

## **CVE-2020-13955 - Apache Calcite (disabled HTTPS hostname verification)**

**Impacted software/package** - Apache Calcite 0.8 to 1.25

**Severity** - **Medium** CVSS: 5.9

**Root-cause** - CWE-295: Improper Certificate Validation leading to CWE-300

### **What is Apache Calcite?**

It's an open-source data management framework used for building databases and data processing systems. It acts as the underlying SQL engine for many massive open-source projects, including Apache Druid and Hive.

### **Vulnerability**

Apache Calcite has a small utility class called **HttpUtils** that it uses to open HTTP and HTTPS connections. The Druid and Splunk adapters rely on it to talk to those services.

The problem is in its **getURLConnection** method. Whenever the connection is HTTPS, the method deliberately switches off the checks that TLS depends on to prove you are actually talking to the real server.

It does two separate things for every HTTPS connection. First, it installs a "trust-all" **SSL socket factory**, which makes the client accept any certificate from anyone, including self signed ones. Second, it sets a **hostname verifier** that always returns true, so the certificate is never matched against the hostname the client is contacting. Put together, these reduce HTTPS to encryption with no authentication of the server, which is exactly the gap a man-in-the-middle attack relies on.

The issue is classified as CWE-295 (Improper Certificate Validation) which could lead to CWE-300 (Man-in-the-Middle attack) and it affects Calcite versions 0.8 through 1.25 and was fixed in 1.26.

### **Man-in-the-Middle (MitM) attacks and their relevance to TLS**

When a client opens an HTTPS connection, TLS gives it an encrypted channel. But encryption on its own is not enough to be safe. You also need to ensure that you are encrypting the traffic to the right server and not to an attacker sitting in between. That assurance comes from the server's certificate, and the client is supposed to check it in two separate ways:

1. Is the certificate signed by a Certificate Authority the client trusts? - This is the trust check, handled by the trust manager.
2. Does the certificate actually belong to the hostname the client tried to reach? - This is hostname verification

Both checks are needed, and the second one is the part Calcite switched off. The trust check on its own only proves that a certificate is genuine and CA-signed. It does not prove that the certificate belongs to the server you meant to connect to. An attacker can easily get a perfectly valid, CA-signed certificate for a domain they own. If the client skips hostname verification, it will happily accept that certificate even though it was issued for the wrong host.

### Objectives of an attacker in such scenario

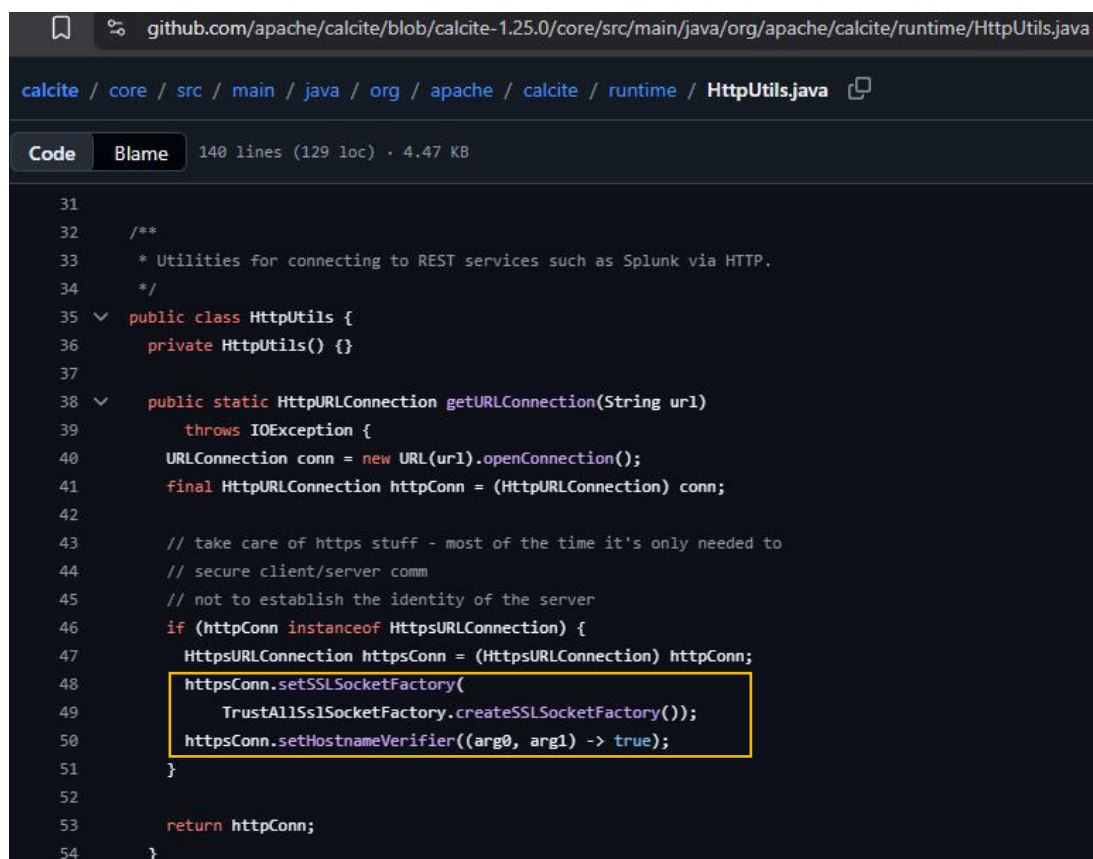
Once positioned in the middle of that connection, the attacker's goals are straightforward.

1. **Read** the traffic. That includes the queries Calcite sends, the data that comes back, and most importantly any credentials or tokens Calcite uses to authenticate to Druid or Splunk.
2. **Modify** the traffic if they want to, altering requests or tampering with the results sent back to the application.
3. **Reuse** any captured credentials to reach the backend data source directly or move further into the environment.

The only prerequisite the attacker needs are a position on the network path. They do not need to compromise a CA, which is what makes a missing hostname check so serious.

### Where it happens in the code

From <https://github.com/apache/calcite/blob/calcite-1.25.0/core/src/main/java/org/apache/calcite/runtime/HttpUtils.java>



```
31
32 /**
33  * Utilities for connecting to REST services such as Splunk via HTTP.
34  */
35 public class HttpUtils {
36     private HttpUtils() {}
37
38     public static HttpURLConnection getUrlConnection(String url)
39         throws IOException {
40         URLConnection conn = new URL(url).openConnection();
41         final HttpURLConnection httpConn = (HttpURLConnection) conn;
42
43         // take care of https stuff - most of the time it's only needed to
44         // secure client/server comm
45         // not to establish the identity of the server
46         if (httpConn instanceof HttpsURLConnection) {
47             HttpsURLConnection httpsConn = (HttpsURLConnection) httpConn;
48             httpsConn.setSSLSocketFactory(
49                 TrustAllSslSocketFactory.createSSLSocketFactory());
50             httpsConn.setHostnameVerifier((arg0, arg1) -> true);
51         }
52
53         return httpConn;
54     }
```

The highlighted two lines do the major damage on the source code.

`setSSLConnectionFactory(TrustAllSslConnectionFactory.createSSLConnectionFactory())` swaps the normal trust logic for one that accepts every certificate, and `setHostnameVerifier((arg0, arg1) -> true)` accepts every hostname.

## How to fix it and prevent it in future

The fix in 1.26 was to remove both overrides and let the JDK defaults handle TLS. The default SSL socket factory validates the certificate against the JVM truststore, and the default hostname verifier checks the hostname. No trust-all factory and no accept-all verifier. Anyone on an affected version should upgrade to 1.26 or later, especially if they use the Druid or Splunk adapters over HTTPS or call HttpUtils directly.

If there was ever a legitimate reason for this, such as an internal server using a self-signed or internal CA certificate, the right way to handle it is to add that specific certificate or CA to a truststore rather than trusting everything. The certificate should also carry the correct hostname so it passes verification normally.

A few things would have stopped this and would catch the next one:

- ✓ Do not disable TLS verification.
- ✓ Keep TLS and other security sensitive settings out of shared utility methods. If a relaxed setting is ever genuinely needed for a local test, scope it to test code and make it obvious, not the default everyone inherits.
- ✓ Prefer HTTP clients that are secure by default, such as `java.net.http.HttpClient` on newer JDKs, instead of hand writing connection helpers that re-implement and weaken TLS handling.
- ✓ Run static analysis. Both of these are easily detected patterns, and SonarQube has dedicated rules for them: one for unverified server certificates (S4830) and one for unverified server hostnames (S5527), both mapping to CWE-295. Reference: <https://github.com/SonarSource/sonar-dotnet/issues/2755>
- ✓ Use dependency scanning so a project still pinned to a vulnerable Calcite version gets flagged for upgrade.
- ✓ In code review, treat any custom `TrustManager`, `SSLConnectionFactory`, or `HostnameVerifier` as something that needs a second look.